

Universidad Panamericana

Especialidad en automatización y control

Comunicación MQTT con ESP32 y Adafruit IO

Curso: FreeRTOS / MICROPROCESADORES

Alumno: Jaime Emmanuel Valenzuela Valdivia

Fecha: 25 de octubre de 2025



Facultad de Ingeniería

Objetivo

Implementar un sistema IoT utilizando un microcontrolador **ESP32** con **FreeRTOS**, capaz de conectarse a una red Wi-Fi y establecer comunicación segura con la plataforma **Adafruit IO** mediante el protocolo **MQTT**. El sistema debe enviar datos simulados de **temperatura** y **ubicación GPS** a la nube para su visualización en tiempo real a través de un dashboard.

Desarrollo

1. Configuración de Red Wi-Fi

Se utilizó una red de 2.4 GHz denominada TP-Link_68CC. El ESP32 fue configurado en modo estación (STA) utilizando las librerías: `esp_wifi.h`, `esp_event.h`, `esp_netif.h` y `nvs_flash.h`.

2. Configuración del Broker MQTT

- Broker: `io.adafruit.com`
- Puerto: 8883 (TLS seguro)
- Usuario: JEV2
- Clave AIO: `aio_...`
- Protocolo: MQTT 3.1.1
- Cliente: ESP32_JEV2

Se utilizó la librería `esp_crt_bundle.h` para validar el certificado TLS y asegurar la conexión cifrada.

3. Publicación de Datos

Se definieron dos tópicos principales:

- **Temperatura:** `JEV2/feeds/temperatura`
- **Ubicación GPS:** `JEV2/feeds/location`

Los datos se enviaron cada 5 segundos desde una tarea FreeRTOS denominada `publisher_task`. Los valores simulados fueron:

- Temperatura: 27.0 °C – 30.0 °C.
- Coordenadas GPS (zona Guadalajara): latitud 20.6736 a 20.6840 °N, longitud -103.3440 a -103.3801 °W.

4. Dashboard en Adafruit IO

En la interfaz web se creó un dashboard titulado **FreeRTOSCourse** con:

- Un medidor tipo gauge para temperatura.
- Un mapa dinámico vinculado al feed de localización.

Ambos bloques muestran actualizaciones en tiempo real provenientes del ESP32.

Resultados

Dato	Feed	Ejemplo publicado	Estado
Temperatura	JEV2/feeds/temperatura	27.0 – 29.9 °C	Correcto
Ubicación GPS	JEV2/feeds/location	20.6736,-103.3440	Correcto
Wi-Fi	TP-Link_68CC	IP: 192.168.0.115	Conectado
TLS	Certificado válido	“Certificate validated”	Seguro
MQTT	Protocolo 3.1.1	Cliente ESP32_JEV2	Estable

Cuadro 1: Datos transmitidos correctamente desde el ESP32 a Adafruit IO.

Durante las pruebas, el sistema publicó datos continuamente sin errores de conexión. Los valores fueron visibles en los feeds de Adafruit IO y graficados en tiempo real.

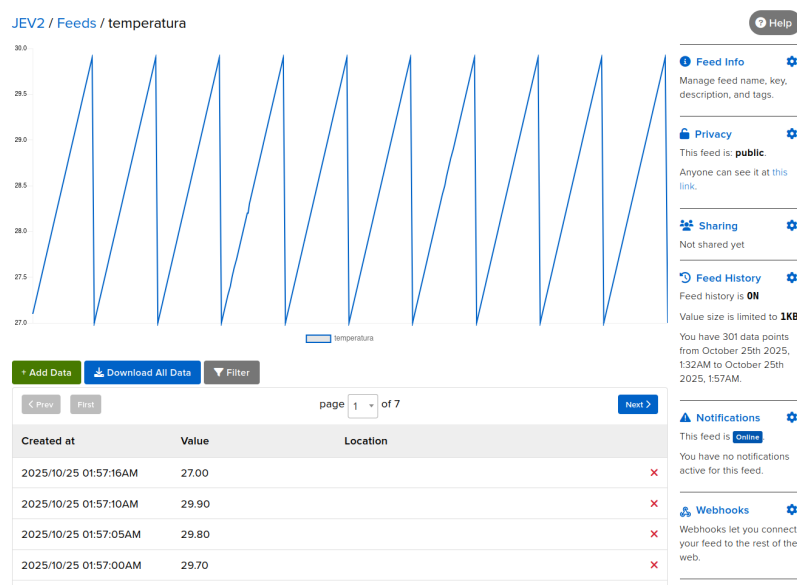


Figura 1: Feed de temperatura mostrando variaciones cíclicas simuladas.

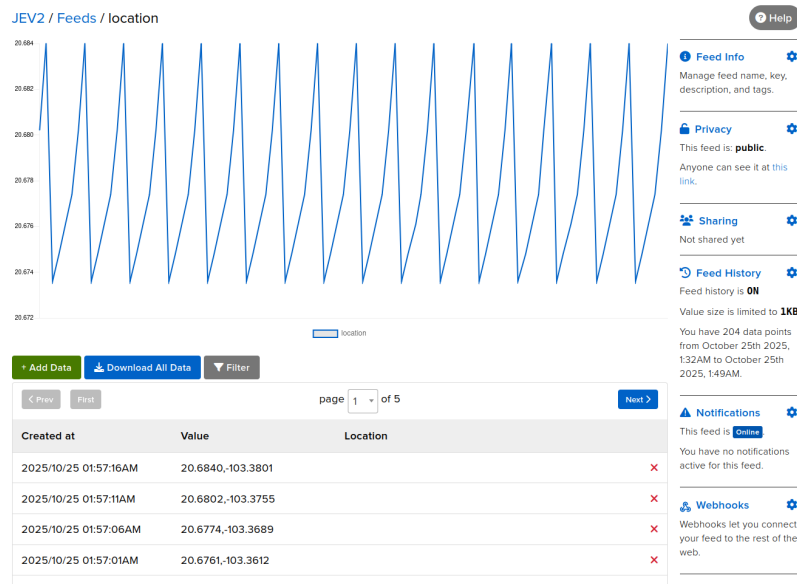


Figura 2: Feed de ubicación GPS con coordenadas cambiantes.

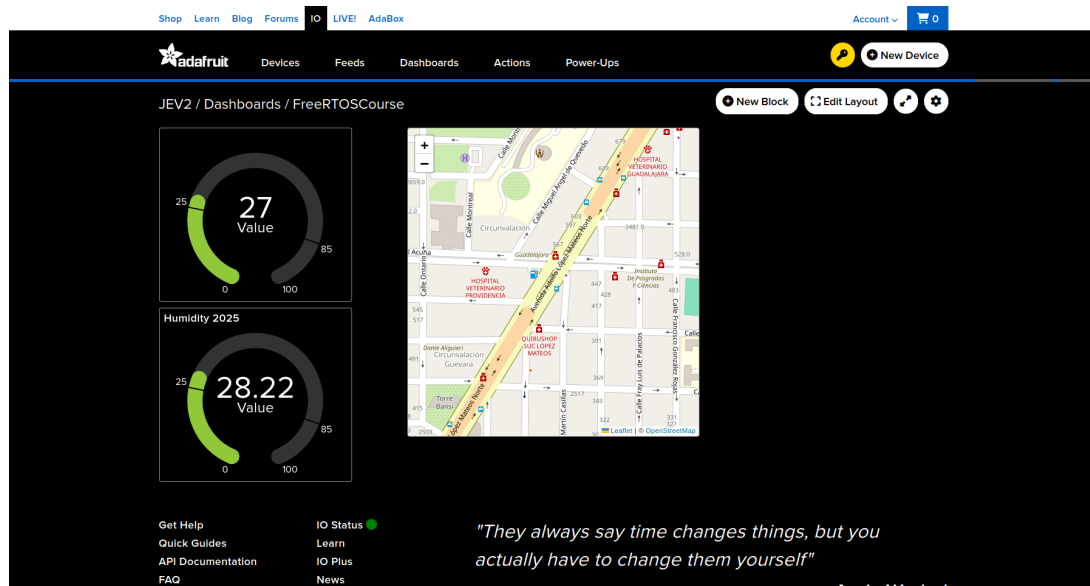


Figura 3: Dashboard en Adafruit IO con medidor y mapa actualizándose en tiempo real.

Evidencia del sistema en ejecución

En la siguiente imagen se muestra la **salida de la terminal en Visual Studio Code**, donde el sistema publica de forma continua los valores de temperatura y coordenadas GPS al broker de Adafruit IO.

```

38 static void wifi_init(void)
39 {
40     ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
41     ESP_ERROR_CHECK(esp_wifi_start());
42     ESP_ERROR_CHECK(esp_wifi_connect());
43     ESP_LOGI(TAG, "Conectando a la red Wi-Fi: %s ...", WIFI_SSID);
44 }
45
46 // ===== MQTT Event Handler ===== */
47 static void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data)
48 {
49     esp_mqtt_event_handle_t event = event_data;
50     client = event->client;
51
52     switch ((esp_mqtt_event_id_t)event_id)
53     {
54         case MQTT_EVENT_CONNECTED:
55             ESP_LOGI(TAG, "Conectado a Adafruit IO (MQTT 3.1.1)");
56             break;
57         case MQTT_EVENT_DISCONNECTED:
58             ESP_LOGI(TAG, "Desconectado del broker. Intentando reconectar...");
59             esp_mqtt_client_reconnect(client);
60             break;
61         case MQTT_EVENT_ERROR:
62             ESP_LOGI(TAG, "Error MQTT");
63             break;
64         default:
65             // No action needed for other events
66     }
67 }
68
69 void app_main()
70 {
71     // Initialize MQTT client
72     esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_config);
73     ESP_ERROR_CHECK(esp_mqtt_client_register(client, mqtt_event_handler));
74     esp_mqtt_client_start(client);
75
76     // Main loop
77     while(1)
78     {
79         // Publish temperature and GPS data
80         publish_data(client);
81         vTaskDelay(pdMS_TO_TICKS(1000));
82     }
83 }

```

Terminal Output:

```

I 1961441 MQTT APP: Publicando GPS: 20.6840,-103.3801
I 1961441 MQTT APP: Publicando temperatura: 20.30 °C
I 1961551 MQTT APP: Publicando GPS: 20.6736,-103.3440
I 1971551 MQTT APP: Publicando temperatura: 20.40 °C
I 1971661 MQTT APP: Publicando GPS: 20.6748,-103.3525
I 1971661 MQTT APP: Publicando temperatura: 20.50 °C
I 1971661 MQTT APP: Publicando GPS: 20.6761,-103.3612
I 1981661 MQTT APP: Publicando temperatura: 20.60 °C
I 1981771 MQTT APP: Publicando GPS: 20.6774,-103.3689
I 1981771 MQTT APP: Publicando temperatura: 20.70 °C
I 1981771 MQTT APP: Publicando GPS: 20.6802,-103.3755
I 1991771 MQTT APP: Publicando temperatura: 20.80 °C
I 1991771 MQTT APP: Publicando GPS: 20.6840,-103.3801
I 1991771 MQTT APP: Publicando temperatura: 20.90 °C
I 1996261 MQTT APP: Publicando GPS: 20.6736,-103.3440
I 1996261 MQTT APP: Publicando temperatura: 20.40 °C
I 1996261 MQTT APP: Publicando GPS: 20.6748,-103.3525
I 1996261 MQTT APP: Publicando temperatura: 20.50 °C
I 1996261 MQTT APP: Publicando GPS: 20.6761,-103.3612
I 1996261 MQTT APP: Publicando temperatura: 20.60 °C
I 1996261 MQTT APP: Publicando GPS: 20.6774,-103.3689
I 1996261 MQTT APP: Publicando temperatura: 20.70 °C
I 1996261 MQTT APP: Publicando GPS: 20.6802,-103.3755
I 1996261 MQTT APP: Publicando temperatura: 20.80 °C
I 1996261 MQTT APP: Publicando GPS: 20.6840,-103.3801
I 1996261 MQTT APP: Publicando temperatura: 20.90 °C

```

Figura 4: Ejecución del código en ESP32 mostrando publicación de datos en tiempo real.

Análisis

El protocolo MQTT demostró ser ligero y eficiente para la transmisión periódica de datos. La conexión mediante TLS añadió una capa de seguridad que garantiza confidencialidad e integridad. El uso de FreeRTOS permitió ejecutar tareas concurrentes sin bloquear la operación general del sistema, mejorando la escalabilidad y robustez.

Conclusiones

- Se estableció con éxito la comunicación entre el ESP32 y Adafruit IO utilizando el protocolo MQTT con autenticación y cifrado TLS.
- Los datos de temperatura y ubicación fueron enviados y visualizados correctamente en la nube.
- El sistema cumple los requerimientos de un entorno IoT básico, demostrando la capacidad del ESP32 para aplicaciones de monitoreo remoto.
- La implementación es estable, segura y fácilmente escalable para futuros proyectos con sensores reales.